

1 Computer 1

1.1 Python

```
重复图片检测完成。  
采样数量：100  
检测耗时：50.64秒  
进程平均CPU：12.89%  
进程峰值CPU：195.30%  
进程平均内存：56.27 MB  
进程峰值内存：64.36 MB
```

Figure 1: 1 image.

```
重复图片检测完成。  
采样数量：100  
检测耗时：50.56秒  
进程平均CPU：8.92%  
进程峰值CPU：208.30%  
进程平均内存：56.61 MB  
进程峰值内存：71.36 MB
```

Figure 2: 4 image.

Table 1: Duplicate-check performance summary

Metric	Mean	Maximum	Final
System CPU (%)	23.10	100.00	100.00
Process CPU (%)	8.92	208.30	97.70
System memory (%)	69.12	69.90	68.90
Process memory (MB)	56.61	71.36	58.28

Runtime: 50.03 s; Images scanned: 10034.

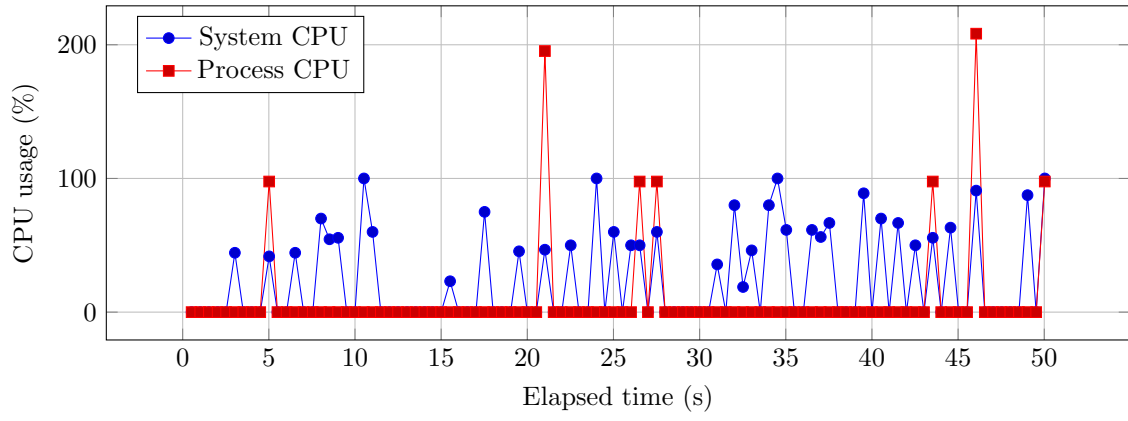


Figure 3: CPU usage during duplicate detection.

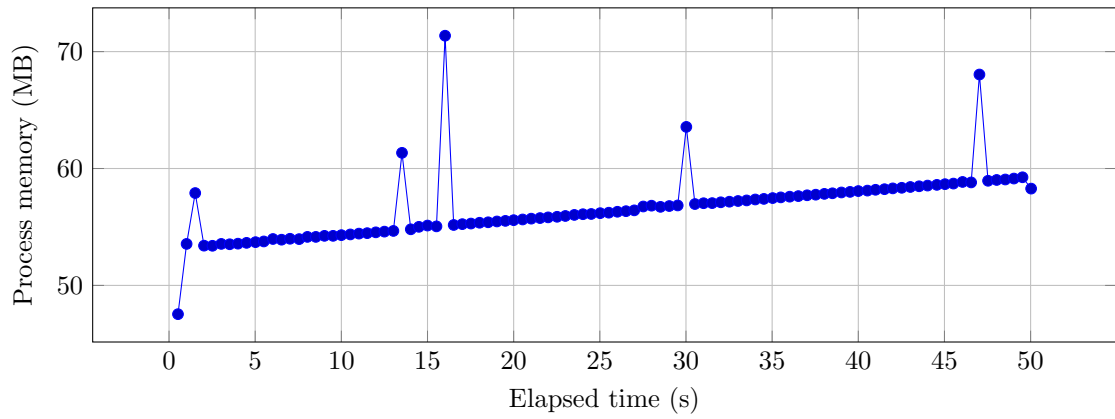


Figure 4: Process memory usage during duplicate detection.

1.2 C++ (CPU only)

1.2.1 1 thread

```
SHA256 actual wall time (file reads excluded): 27416.999 ms
Byte comparison actual wall time (file reads excluded): 11.763 ms
SHA256 + duplicate comparison actual wall time: 27428.762 ms (27.429 s)
=== Completed: existing duplicates found ===
```

Figure 5: Duplicate-detection timing results for the CPU implementation with 1 CPU worker thread.

Table 2: CPU and memory usage summary for the CPU-only 1-thread test

Metric	Mean	Maximum	Final
System CPU (%)	35.59	100.00	32.20
Process CPU (%)	81.09	117.60	66.50
System memory (%)	52.98	53.30	52.90
System memory used (MB)	16983.75	17100.53	16955.25
Process memory RSS (MB)	10.05	12.10	11.02
Process memory VMS (MB)	5.23	7.26	6.25
Process threads	3.00	5	1

Process: Project1.exe; PID: 147320; runtime: 89.83 s; samples: 893.

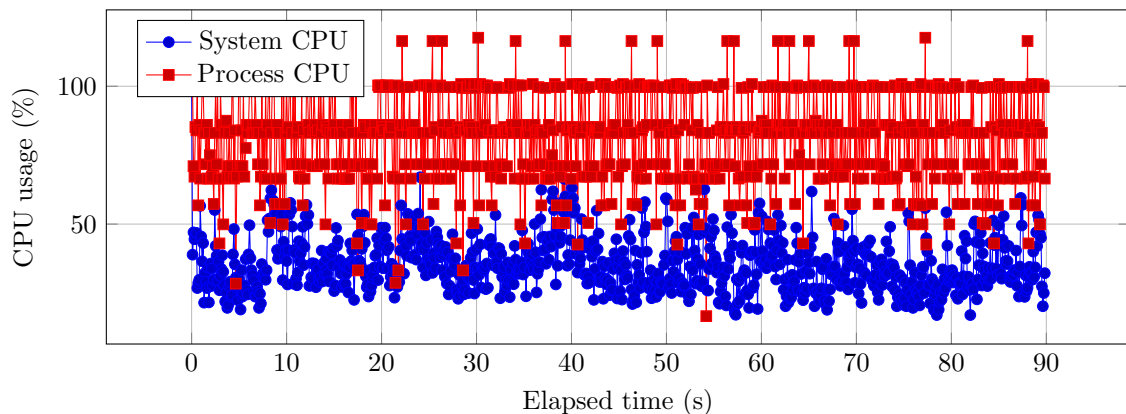


Figure 6: CPU usage during the CPU-only 1-thread program execution.

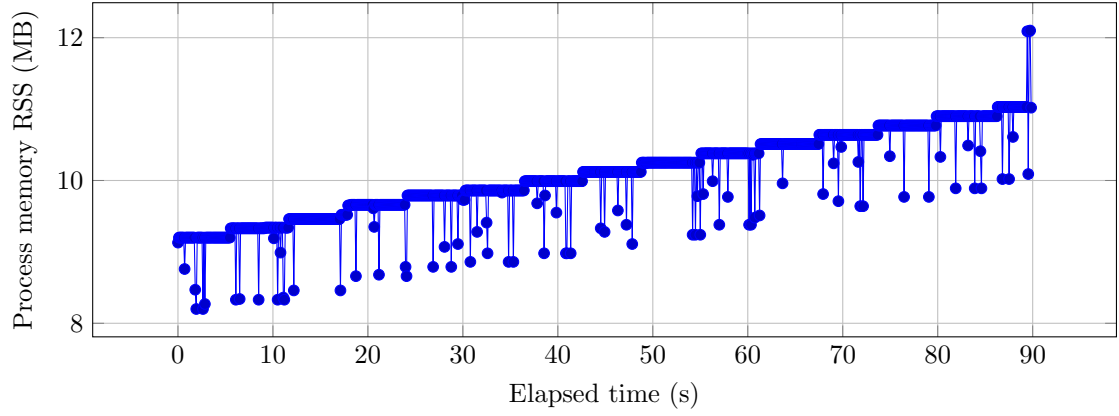


Figure 7: Process physical-memory usage during the CPU-only 1-thread program execution.

1.2.2 8 threads

```
SHA256 compute only (file reads excluded): 28993.890 ms
Byte comparison only (file reads excluded): 10.399 ms
SHA256 + duplicate comparison compute only: 29004.289 ms (29.004 s)
```

Figure 8: Duplicate-detection timing results for the CPU implementation with 8 CPU worker threads.

Table 3: CPU and memory usage summary for the CPU-only 8-threads test

Metric	Mean	Maximum	Final
System CPU (%)	71.00	95.20	35.60
Process CPU (%)	485.54	648.30	100.30
System memory (%)	52.90	54.20	51.80
System memory used (MB)	16959.15	17359.87	16604.46
Process memory RSS (MB)	17.35	18.77	12.71
Process memory VMS (MB)	12.74	13.99	7.80
Process threads	11.75	12	4

Process: **Project1.exe**; PID: 146908; runtime: 17.18 s; samples: 167.

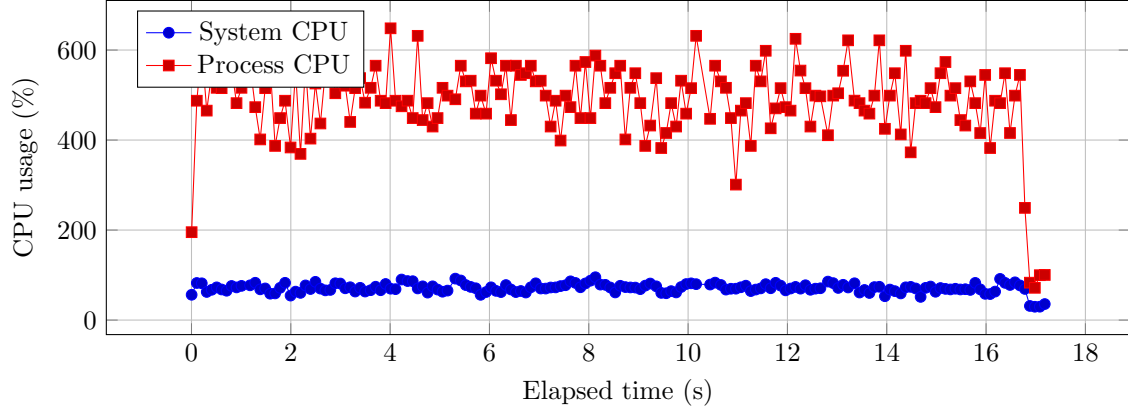


Figure 9: CPU usage during the CPU-only 8-threads program execution.

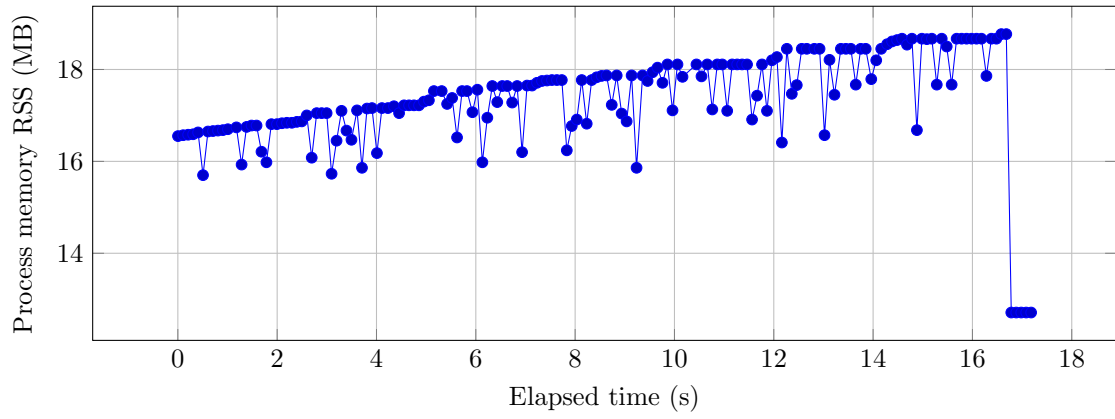


Figure 10: Process physical-memory usage during the CPU-only 8-threads program execution.

1.2.3 16 threads

```
SHA256 compute only (file reads excluded): 31031.829 ms
Byte comparison only (file reads excluded): 11.306 ms
SHA256 + duplicate comparison compute only: 31043.136 ms (31.043 s)
```

Figure 11: Duplicate-detection timing results for the CPU implementation with 16 CPU worker threads.

Table 4: CPU and memory usage summary for the CPU-only 16-threads test

Metric	Mean	Maximum	Final
System CPU (%)	87.10	100.00	22.90
Process CPU (%)	652.98	900.00	100.80
System memory (%)	53.22	54.60	52.10
System memory used (MB)	17061.08	17492.39	16691.97
Process memory RSS (MB)	25.30	27.52	11.61
Process memory VMS (MB)	20.83	22.88	7.34
Process threads	19.01	20	4

Process: `Project1.exe`; PID: 147176; runtime: 15.49 s; samples: 97.

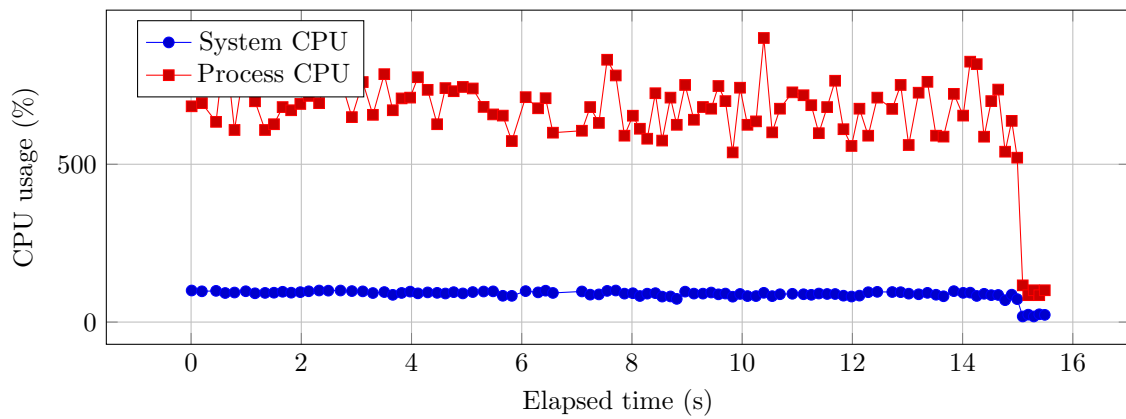


Figure 12: CPU usage during the CPU-only 16-threads program execution.

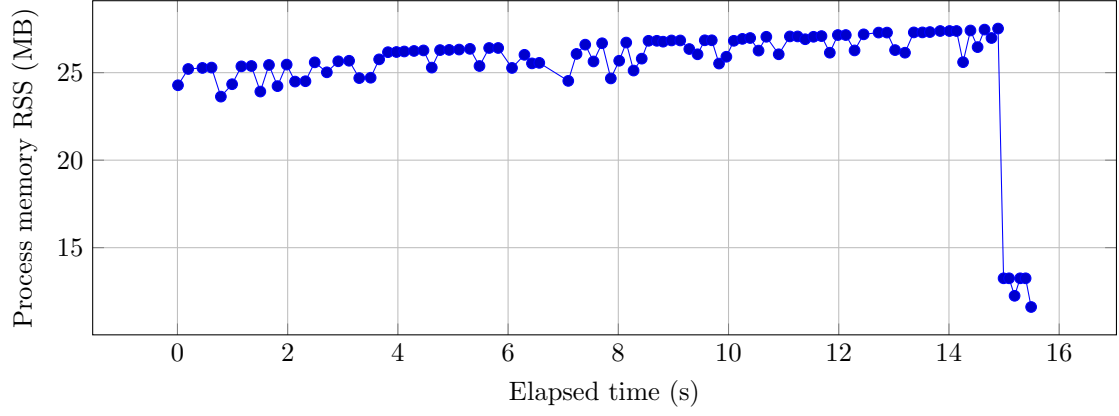


Figure 13: Process physical-memory usage during the CPU-only 16-threads program execution.

1.3 C++ (GPU)

1.3.1 1 thread

```
[INDEX READY] 10067 images | duplicates 64 | SHA256 compute cumulative (reads excluded) 1170.56 ms | byte comparison compute cumulative (reads excluded) 5.98 ms | SHA256 + duplicate comparison compute cumulative 1176.54 ms (1.18 s) | full index total 21409.78 ms
```

Figure 14: Duplicate-detection timing results for the GPU implementation with 1 CPU worker thread.

Table 5: CPU-side resource usage summary for the GPU implementation with 1 CPU worker thread

Metric	Mean	Maximum	Final
System CPU (%)	40.11	70.40	55.30
Process CPU (%)	65.16	116.40	83.10
System memory (%)	55.11	56.70	55.10
System memory used (MB)	17669.26	18180.76	17670.02
Process memory RSS (MB)	555.16	643.91	623.66
Process memory VMS (MB)	1113.05	1288.80	1194.15
Process threads	6.61	7	6

Process: `Project1.exe`; PID: 66200; runtime: 21.42 s; samples: 212.

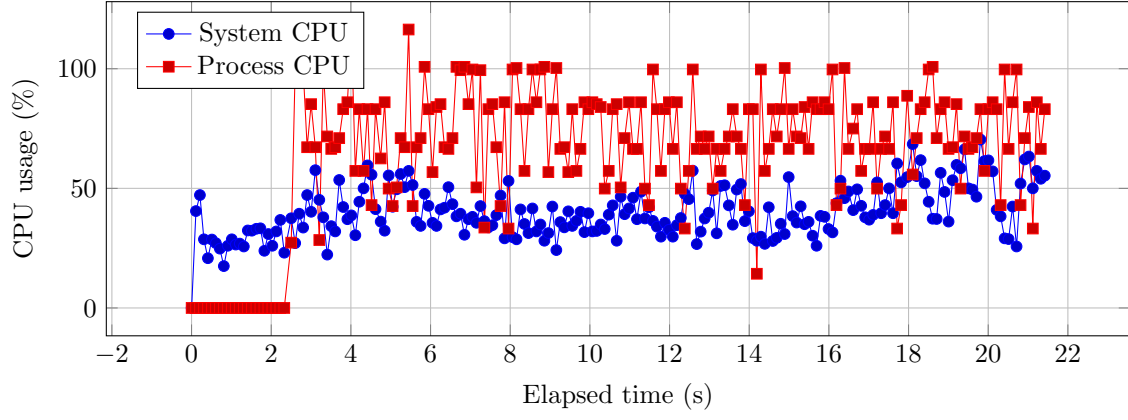


Figure 15: CPU usage during execution of the GPU implementation with 1 CPU worker thread.

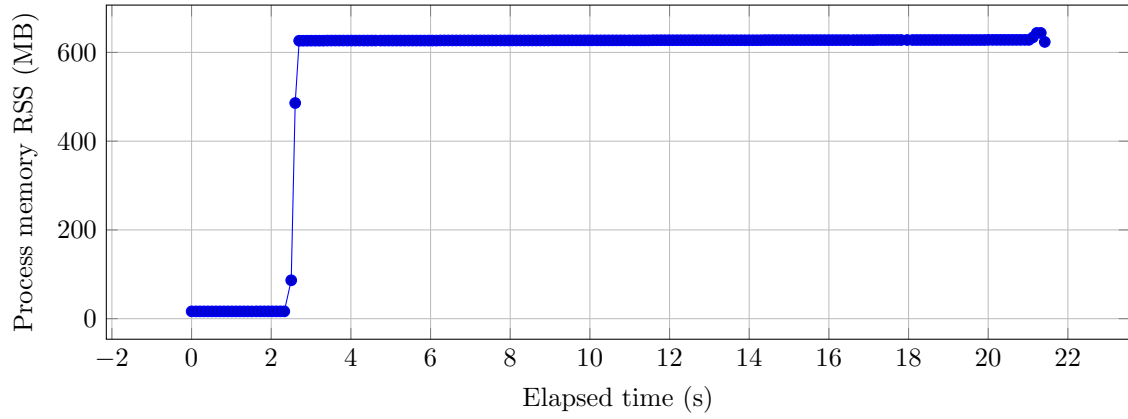


Figure 16: Process physical-memory usage during execution of the GPU implementation with 1 CPU worker thread.

1.3.2 8 threads

```
[INDEX READY] 10067 images | duplicates 64 | SHA256 actual wall (reads excluded) 673.23 ms | byte comparison actual wall (reads excluded) 5.38 ms | SHA256 + duplicate comparison actual wall 678.61 ms (0.68 s) | full index total 9177.49 ms
```

Figure 17: Duplicate-detection timing results for the GPU implementation with 8 CPU worker threads.

Table 6: CPU-side resource usage summary for the GPU implementation with 8 CPU worker threads

Metric	Mean	Maximum	Final
System CPU (%)	78.34	100.00	54.10
Process CPU (%)	464.72	731.40	149.60
System memory (%)	55.39	58.10	56.60
System memory used (MB)	17755.90	18632.65	18145.20
Process memory RSS (MB)	517.30	644.50	644.50
Process memory VMS (MB)	1034.23	1289.00	1288.97
Process threads	11.32	14	6

Process: Project1.exe; PID: 132420; runtime: 12.69 s; samples: 104.

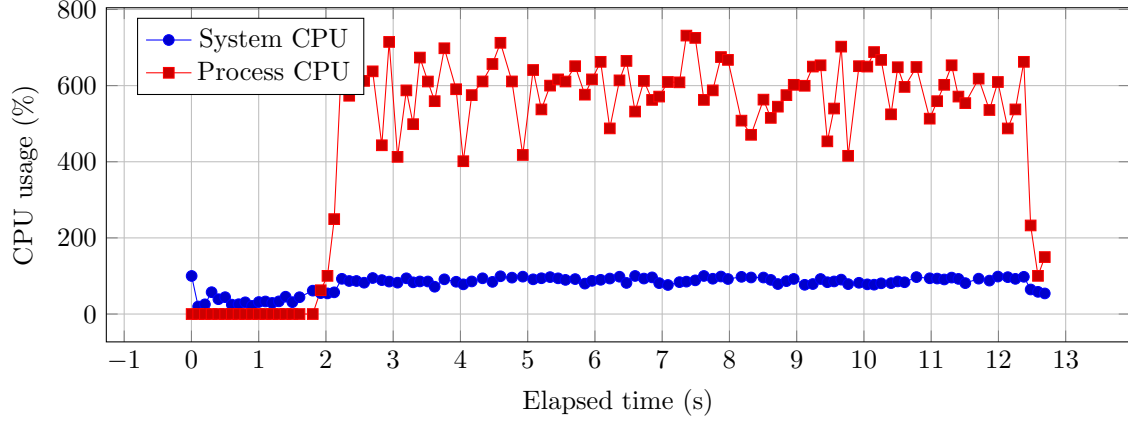


Figure 18: CPU usage during execution of the GPU implementation with 8 CPU worker threads.

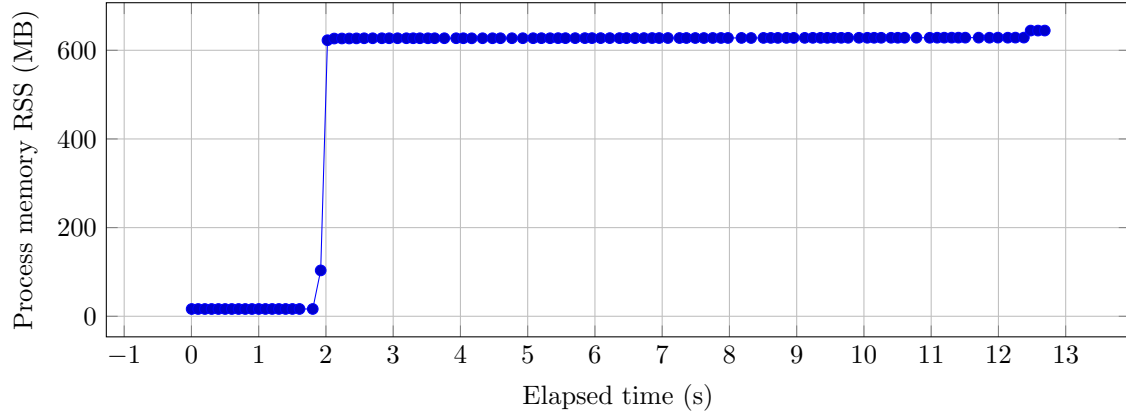


Figure 19: Process physical-memory usage during execution of the GPU implementation with 8 CPU worker threads.

1.3.3 16 threads

```
[INDEX READY] 10067 images | duplicates 64 | SHA256 actual wall (reads excluded) 695.96 ms | byte comparison actual wall (reads excluded) 5.84 ms | SHA256 + duplicate comparison actual wall 701.80 ms (0.70 s) | full index total 8688.53 ms
```

Figure 20: Duplicate-detection timing results for the GPU implementation with 16 CPU worker threads.

Table 7: CPU-side resource usage summary for the GPU implementation with 16 CPU worker threads

Metric	Mean	Maximum	Final
System CPU (%)	76.10	100.00	28.20
Process CPU (%)	584.30	1183.00	99.40
System memory (%)	55.64	58.10	57.60
System memory used (MB)	17836.80	18613.74	18471.91
Process memory RSS (MB)	482.87	645.56	645.56
Process memory VMS (MB)	962.78	1291.04	1291.01
Process threads	14.94	22	6

Process: `Project1.exe`; PID: 144268; runtime: 14.50 s; samples: 88.

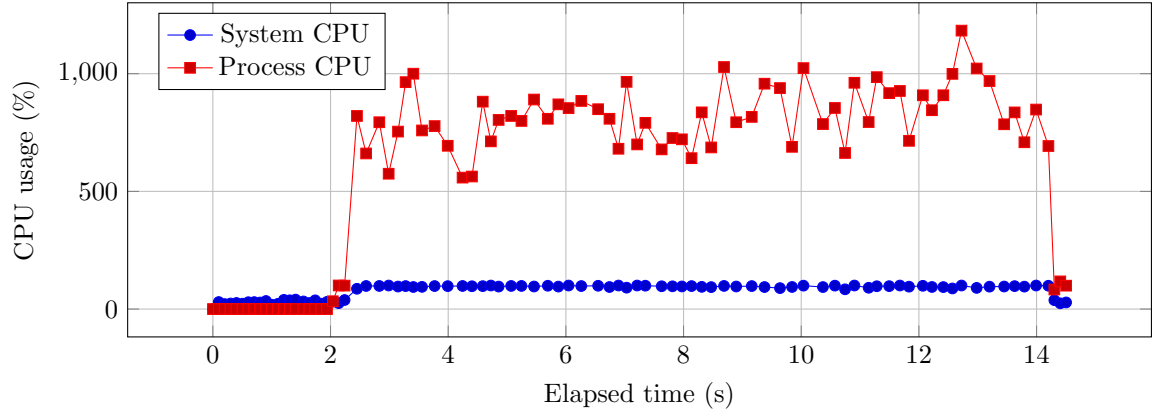


Figure 21: CPU usage during execution of the GPU implementation with 16 CPU worker threads.

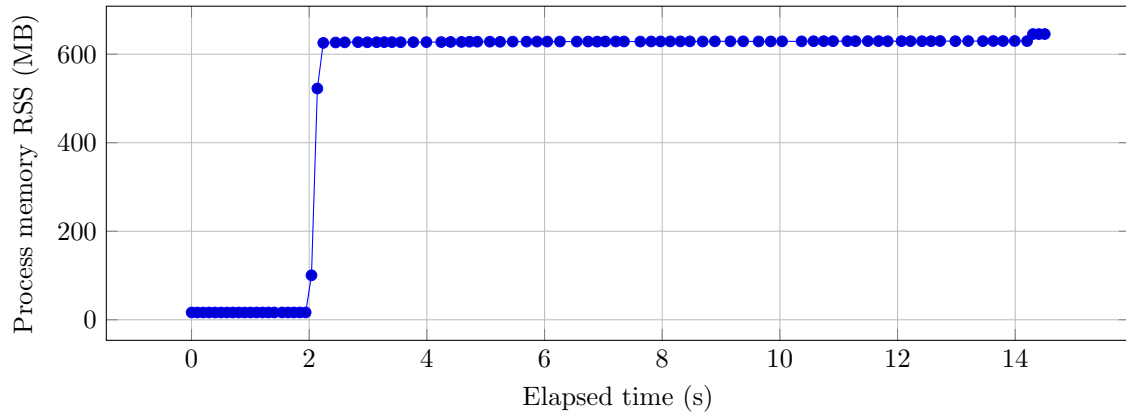


Figure 22: Process physical-memory usage during execution of the GPU implementation with 16 CPU worker threads.

2 Computer 2

2.1 Python

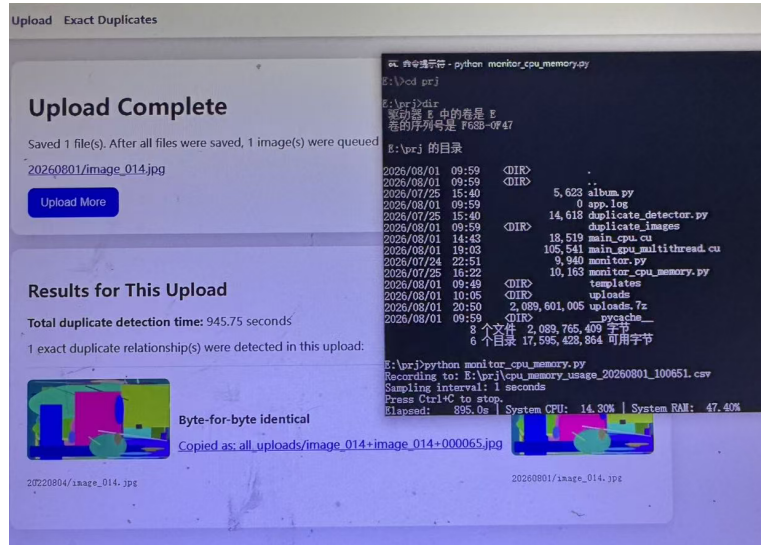


Table 8: System CPU and memory usage summary

Metric	Mean	Minimum	Maximum	Final
System CPU (%)	20.12	4.80	62.10	12.70
System memory (%)	48.17	45.80	51.90	47.30
System memory used (MB)	7799.90	7422.50	8410.64	7656.26
System memory available (MB)	8392.26	7781.52	8769.66	8535.90

Monitoring duration: 913.00 s; samples: 914.

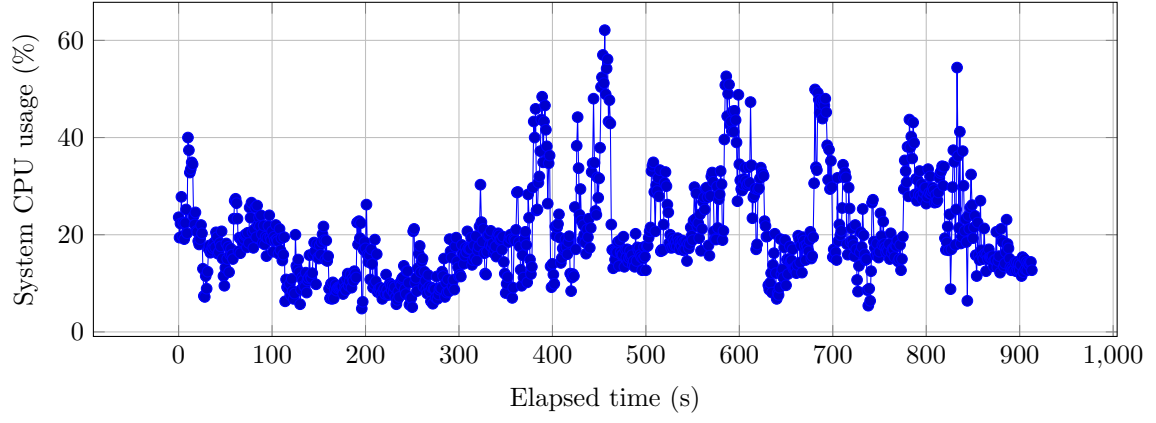


Figure 23: System CPU usage during monitoring (Computer 2).

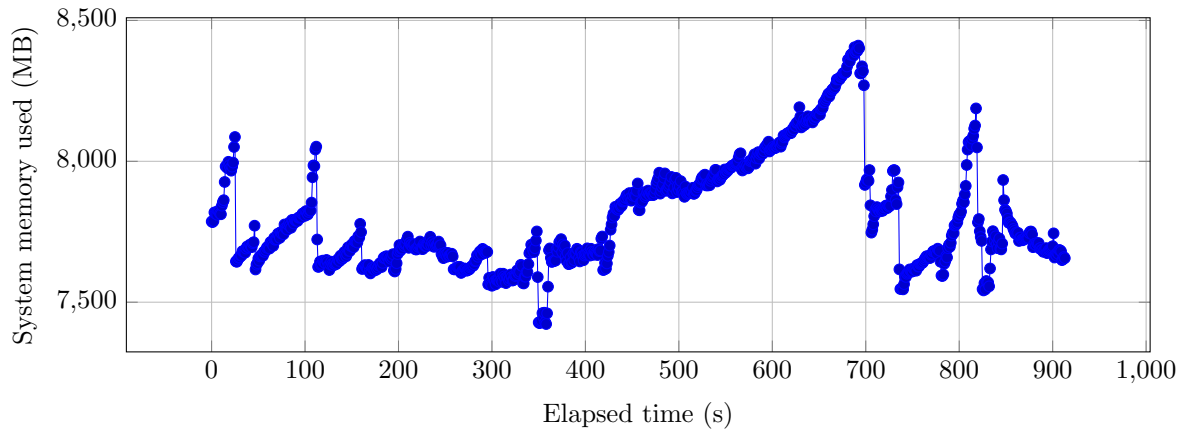


Figure 24: System memory usage during monitoring (Computer 2).

2.2 C++ (CPU only)

2.2.1 1 thread

Table 9: System CPU and memory usage summary

Metric	Mean	Minimum	Maximum	Final
System CPU (%)	11.34	0.00	50.40	1.80
System memory (%)	49.76	45.90	51.70	45.90
System memory used (MB)	8057.26	7426.55	8378.25	7428.46
System memory available (MB)	8134.90	7813.91	8765.61	8763.70

Monitoring duration: 544.20 s; samples: 395.

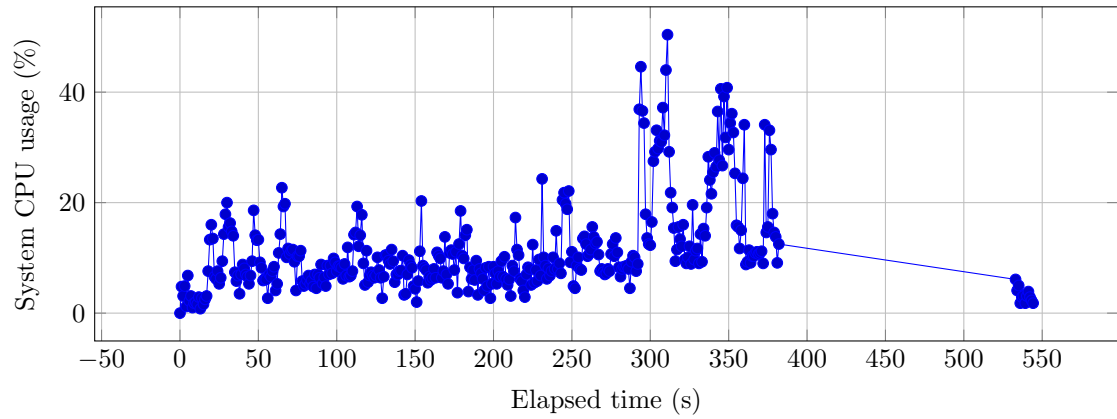


Figure 25: System CPU usage during monitoring (Computer 2).

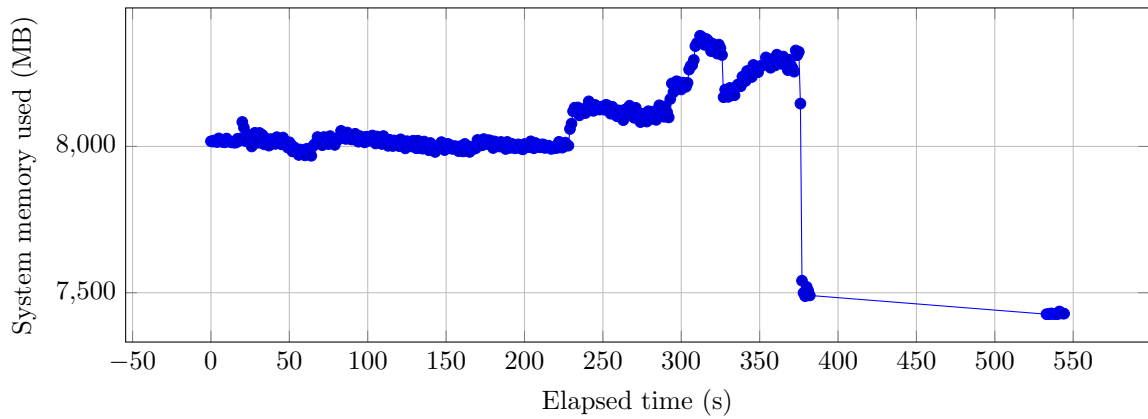


Figure 26: System memory usage during monitoring (Computer 2).

2.3 C++ (GPU)

2.3.1 8 threads

Table 10: System CPU and memory usage summary

Metric	Mean	Minimum	Maximum	Final
System CPU (%)	9.68	0.00	27.10	1.20
System memory (%)	54.89	50.00	57.50	50.10
System memory used (MB)	8887.92	8098.40	9316.04	8105.38
System memory available (MB)	7304.24	6876.12	8093.76	8086.78

Monitoring duration: 643.00 s; samples: 644.

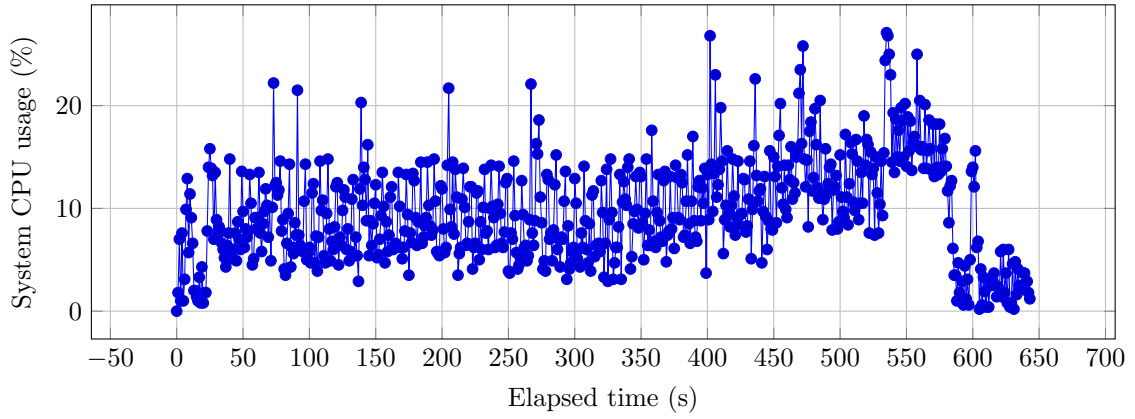


Figure 27: System CPU usage during monitoring (Computer 2).

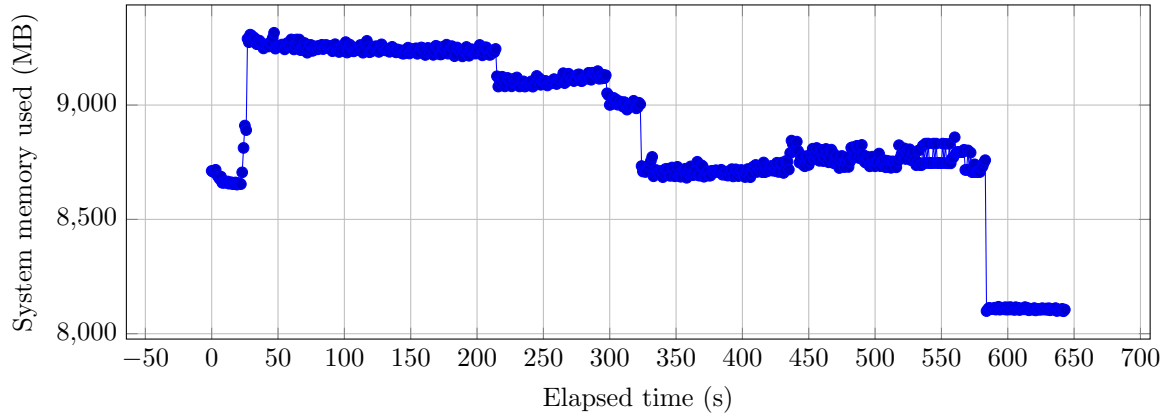


Figure 28: System memory usage during monitoring (Computer 2).

3 Experimental Environment

The environment of computer 1 is shown in Table 11.

Table 11: Experimental environment (computer 1)

Component	Configuration
Operating system	Windows 11 64-bit
CPU	AMD Ryzen 7 8845HS w/ Radeon 780M Graphics
System memory	32 GB
GPU	NVIDIA NVIDIA GeForce RTX 4070
GPU memory	8 GB
Dataset storage	SSD
NVIDIA driver	577.02
CUDA Toolkit	CUDA 12.9
Python	Python 3.11.9
C++ compiler	Microsoft Visual C++ 19.44.35228
C++ standard	C++17
Build configuration	Release, x64, optimization enabled

The environment of computer 2 is shown in Table 12.

Table 12: Experimental environment (Computer 2)

Component	Configuration
Operating system	Windows 10 64-bit
CPU	Intel Core i5-8265U @ 1.60 GHz
System memory	16 GB
GPU	NVIDIA GeForce MX150
GPU memory	2 GB
Dataset storage	HDD
NVIDIA driver	N/A
CUDA Toolkit	CUDA 11.6
Python	3.11.9
C++ compiler	VC2019
C++ standard	C++17
Build configuration	Debug, x64

4 Conclusion

This work compared Python, CPU-only C++, and GPU-accelerated C++ implementations for duplicate-image detection. On Computer 1, CPU multithreading reduced the total runtime from 89.83 s with one thread to 15.49 s with 16 threads.

The GPU implementation achieved substantially shorter computation times. The measured time was 1.026 s with one CPU worker thread, 0.679 s with 8 threads, and 0.702 s with 16 threads. At the same thread counts of 1, 8, and 16, GPU acceleration improved the SHA-256 hashing and duplicate-comparison performance by approximately 26.7 $\times$, 42.7 $\times$, and 44.2 $\times$, respectively, compared with the CPU-only implementation. The GPU configuration with 8 CPU worker threads provided the best result. It was The slight increase from 8 to 16 threads suggests that additional thread scheduling and synchronization overhead limited further improvement.

The current work is using NVIDIA Nsight Systems and NVIDIA Nsight Compute to analyze the CPU-GPU execution pipeline and individual CUDA kernel performance. Future work will focus on optimizing CUDA kernels, memory transfers, GPU occupancy, buffer reuse, and the overlap among file reading, data transfer, and GPU computation.

Table 13: Summary of duplicate-detection performance on Computer 1

Implementation	Threads	SHA-256 + Comparison (s)	Total Runtime (s)	Mean Process CPU (%)	Mean Process Memory (MB)
Python	–	50.030	50.03	8.92	56.61
CPU-only	1	27.429	89.83	81.09	10.05
CPU-only	8	29.004	17.18	485.54	17.35
CPU-only	16	31.043	15.49	652.98	25.30
GPU	1	1.026	21.42	65.16	555.16
GPU	8	0.679	12.69	464.72	517.30
GPU	16	0.702	14.50	584.30	482.87

Note: When using multiple threads, **SHA-256 + Comparison** represents the cumulative computation time of all threads, so it may be greater than the actual elapsed wall-clock time.

Table 14: System resource monitoring results on Computer 2

Implementation	Monitoring Time (s)	Mean System CPU (%)	Mean Memory Used (MB)
Python	913.00	20.12	7799.90
CPU-only	544.20	11.34	8057.26
GPU (8 threads)	643.00	9.68	8887.92